

视频课程笔记

AI 模型的思考杠杆

The Thinking Lever

基于 Bilibili 视频、AI 字幕与逐帧画面复核整理



Bilibili 标题：AI 模型的思考杠杆

本地上传者：TristanBot

原讲者：Alexander Bricken, Anthropic Applied AI Product Engineer

发布日期：2026-05-21

视频时长：21:17

视频链接：<https://www.bilibili.com/video/BV1awLt6zE8f/>

字幕说明：优先探测英文字幕，但 Bilibili 仅暴露 ai-zh 字幕轨道；正文以该字幕为时间轴，并用关键帧复核可见内容。

报告版本：v1

阅读导引

这份笔记围绕一个实际工程问题展开：当 Claude 面对难任务时，开发者应该给它多少运行时思考空间。视频的答案先承认“多给 token 会更强”这一直觉，再把问题拆成模型选择、努力等级、思考预算、工具使用、延迟、成本和质量线之间的组合决策。

读者可以把“思考杠杆”理解为一套运行时资源控制面。训练时计算像模型出厂前积累的经验，测试时计算像模型接到任务后的草稿纸、推演时间和工具调用空间。草稿纸变多通常会改善困难任务上的表现，也会带来更长等待、更高 token 消耗和更高成本。真正的工程判断发生在这三者之间：什么任务值得多想，什么任务应当快速完成，什么任务需要换更强模型。

核心结论

Claude 的思考能力应当被当成可调资源管理：保留思考、工具调用和最终输出三种能力，再用努力等级、令牌预算、任务预算和评估集来约束它。粗暴关闭思考会削掉模型能力；无限提高努力等级也会把成本和延迟推高。更稳的路径是先设定质量线，再用真实任务评估选择模型与努力等级。

目录

阅读导引	1
1 为什么思考是一根杠杆	3
1.1 问题从哪里来：快速回答会留下性能空间	3
1.2 训练时计算与测试时计算：同一条上升直觉	4
1.3 为什么 token 能换来质量	4
1.4 证据范围：图表之外还有多类困难任务	5
1.5 本章小结	5
2 交通灯模拟：投入强度如何改变结果	6
2.1 把抽象杠杆变成一个固定任务	6
2.2 低投入：基本结构成立，但细节较薄	6
2.3 高投入：质量提升首先体现在约束处理	6
2.4 最大投入：细节密度上升，成本也突然变大	8
2.5 一个抽象公式：投入、质量、时间一起移动	8
2.6 从几分钟仿真到长时程自主工作	9
2.7 本章小结	9
3 测试时计算的三种载体	10
3.1 把“多想一会儿”拆成可管理的资源	10
3.2 思考：模型的草稿纸和推理空间	11
3.3 工具使用：连接外部世界的接口	11

3.4	最终文本：把工作结果交给用户	12
3.5	努力程度与预算：软旋钮和硬边界	12
3.6	本章小结	13
4	从扩展思考到交错思考，再到自适应思考	13
4.1	理想状态：模型自己判断该投入多少	13
4.2	扩展思考：先集中推理，再执行工具	14
4.3	交错思考：工具之后还要重新推理	15
4.4	自适应思考：把何时思考交给模型判断	15
4.5	效率：为什么自适应思考可能更接近帕累托最优	17
4.6	本章小结	18
5	思考开关的误区：把能力留给模型	18
5.1	把思考当开关，会丢失控制精度	18
5.2	工具类比：让能力可用，再让模型判断何时使用	19
5.3	同事类比：给约束，让对方完成分析	19
5.4	为下一步做准备：从能力开关走向评估校准	20
5.5	本章小结	20
6	用评估校准努力等级	20
6.1	真实难题才会暴露投入差异	20
6.2	边际收益递减：最高档经常很贵	21
6.3	低努力的价值：速度约束会改变搜索路径	22
6.4	Pokemon Red 例子：低努力带来的捷径策略	22
6.5	怎样建立可用的努力等级评估	23
6.6	本章小结	23
7	模型大小与努力程度的组合决策	23
7.1	从完美评估退一步：先用经验法则落地	23
7.2	努力等级阶梯：从最大努力到低努力	24
7.3	小模型高努力，还是大模型低努力	25
7.4	组合决策表：从质量线反推资源配置	26
7.5	本章小结	27
8	总结与延伸：从手动杠杆到预算化任务	27
8.1	演讲者的最后清单：把思考能力留给模型	27
8.2	决策框架：质量线、预算、模型、工具	28
8.3	预算化长期任务：从一次回答到一段工作	28

8.4 给读者的落地流程	29
8.5 本章小结	30
资料来源与产物说明	30

1 为什么思考是一根杠杆

1.1 问题从哪里来：快速回答会留下性能空间

演讲从一个很直接的问题开始：Claude 面对复杂任务时，是否应当在回答前多花一些运行时计算。00:00:28–00:00:41 中，讲者把主题称为“The thinking level”，中文讲义中统一写作“思考杠杆”。这里的“杠杆”带有明确工程含义：它指一个可以调节的运行时变量。模型在推理阶段多使用 token，就可能把更多中间分析、约束检查和候选路径放进工作过程里。



图 1：开场标题页展示“The thinking level”，把本节主题定位为 Claude 如何把运行时思考变成可调节的杠杆。¹

字幕 1–16 中有几句铺垫可以直接压缩成一个教学目标：讲者要讨论 Claude 在运行与推理时如何利用计算资源，尤其是如何用更多 token 解决更棘手的复杂问题。这里的“token”可以理解为模型写在草稿纸上的最小语言单位。人类做难题时会先列条件、画图、验算、排除错误路线；模型多花 token 时，也是在给这些中间步骤留下空间。

本节的核心定义

测试时计算指模型已经训练完成之后，在一次具体任务运行期间额外消耗的计算。它常表现为更多推理 token、更长等待时间和更多中间检查。思考杠杆就是把这部分运行时投入作为一个可调变量，用来换取更高的任务质量。

这个定义需要和训练阶段区分开。训练阶段像把知识、模式和能力压进模型参数；测试阶段像让已经训练好的模型在具体题目上多走几步。前者改变模型本身，后者改变这一次回答的工作量。讲者的论点是：二者都能提高表现，只是发生在不同时间点。

1.2 训练时计算与测试时计算：同一条上升直觉

00:01:02–00:01:24 说明，大语言模型过去几年的重要变化之一，是推理模型开始显式利用测试时计算。字幕说得很朴素：投入更多 token，让模型更有效地回答问题；训练计算与测试时计算同样能带来更高的智能水平。这里的“同样”需要谨慎理解。它并不表示两者机制完全相同，只表示从结果曲线上看，增加计算投入都可能带来性能提升。

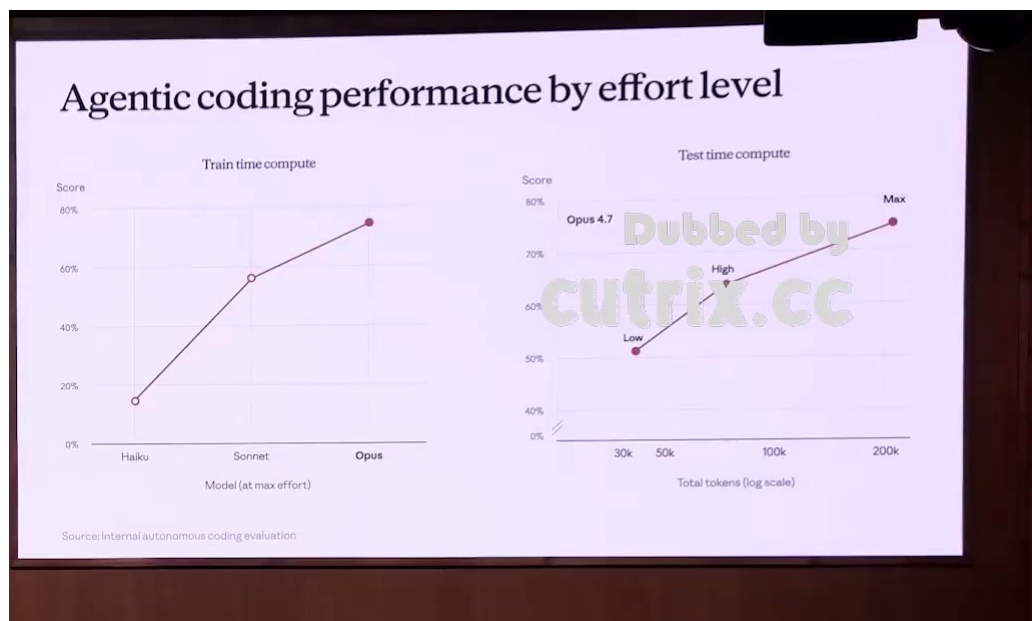


图 2：图中左侧比较 Haiku、Sonnet、Opus 等模型规模对应的训练时计算效果，右侧展示 Opus 4.7 在 Low、High、Max 不同投入下，随着总 token 增加，智能体编码任务分数上升。²

图 2 是本节最关键的证据。左侧图把模型家族从 Haiku 到 Sonnet 再到 Opus 的能力提升放在训练时计算维度上；右侧图固定在 Opus 4.7，观察 Low、High、Max 三档运行时投入。字幕 24–38 也对应这层解释：左侧显示模型规模增加时性能稳步攀升，右侧使用对数坐标显示 Claude 投入更多 token 后实际性能提升。

怎样读这张图

左图回答“换更强模型会怎样”，右图回答“同一模型多花运行时 token 会怎样”。两条路线都能推高任务分数。工程上真正要做的选择，是在模型大小、token 消耗、等待时间和质量要求之间找平衡。

视频没有要求读者记住某个精确斜率，也没有给出“token 增加多少就提升多少分”的定律。更稳妥的理解是方向性的：当任务足够难，模型多花一点运行时计算，经常能减少仓促回答带来的遗漏。

1.3 为什么 token 能换来质量

从机制上看，token 的作用可以拆成三层。第一层是展开问题：模型把模糊请求拆成条件、目标和限制。第二层是搜索路径：模型尝试多个解法或多个解释角度。第三层是检查输出：模型在给出最终答案前发现冲突、补齐遗漏、修正局部错误。字幕 34–46 中的说法是，随着 Claude 投入更多 token，实际性能随之提升；在多个测试结果中，回答前投入更多思考，最终效果会更好一些。

可以用一个很小的抽象式表达这层关系。它只用于讲义解释，视频并没有把它作为测量公式：

$$Q \approx f(M, B_{\text{tok}}, \text{task}, \text{context})$$

- Q : 输出质量或任务成功率，例如编码任务分数、仿真完成度、复杂问答正确率。
- M : 模型选择，例如 Haiku、Sonnet、Opus 这类模型家族。
- B_{tok} : 本次运行允许消耗的 token 空间，包含推理和输出相关的 token。
- task : 任务本身的难度与结构。
- context : 模型可见的上下文，包括提示词、资料、代码、界面状态等。

这个式子想表达的只有一件事：质量由多项变量共同决定。更多 token 有帮助，前提是任务值得多想，模型也有能力把额外 token 用在有效推理上。若题目很简单，额外 token 可能只会增加等待；若上下文缺失，模型多想也可能围着错误信息打转。

不要把方向性证据读成万能规律

本节的图和字幕支持“更多测试时计算常能改善困难任务表现”。它没有支持线性提升、必然提升，也没有支持所有任务都应使用最高投入。后续章节会继续讨论成本、延迟和如何选择投入等级。

1.4 证据范围：图表之外还有多类困难任务

00:02:12–00:02:36 中，字幕把证据范围从智能体编码扩展到其他任务族：计算机使用基准、类似真实世界任务的评测，以及 Human's Last Exam 这类博士水平问题。图像报告说明，这个窗口没有可读的基准列表画面，所以讲义只保留字幕层面的概括，不把它扩写成完整基准分类。

这段话的教学价值在于建立“难题才显出杠杆”的直觉。简单分类题或短摘要题可能不需要长时间推理；跨工具编码、计算机操作、博士水平问答这类任务，往往包含更多约束和更长因果链。模型若一开始就急着给最终答案，容易把中间检查省掉。多花 token 的收益，通常出现在这些需要计划、验证和修正的地方。

1.5 本章小结

本章覆盖 00:00:18–00:02:38 的开场论证。思考杠杆的核心，是把测试时计算视为可调节的运行投入：同一模型在回答前多消耗 token，常能在困难任务上取得更好结果。训练时计算像提升模型底座，测试时计算像让模型在当前题目上多走几步；两者都可能提高表现，但成本和使用方式不同。

图 2 给出本节最具体的证据：模型规模提升与运行时 token 增加都对应更高的智能体编码分数。字幕还把这一直觉扩展到计算机使用、真实世界任务和博士水平问题。需要保留的边界也很清楚：这些证据是方向性的，不等于所有任务都要最大投入。下一章会用交通灯仿真把这根杠杆的收益和代价放到同一个可见例子里。

2 交通灯模拟：投入强度如何改变结果

2.1 把抽象杠杆变成一个固定任务

上一章讲的是图表和基准，本章把同一件事放到一个实时演示里。00:02:39–00:03:08 中，讲者说 Anthropic 的人当然喜欢看图表，但他更想展示一个实际运行效果。控制变量很清楚：提示词保持一致，只改变投入强度，观察 Claude 生成的交通灯仿真怎样变化。这里的“投入强度”，就是后文正式定义的“努力等级”。

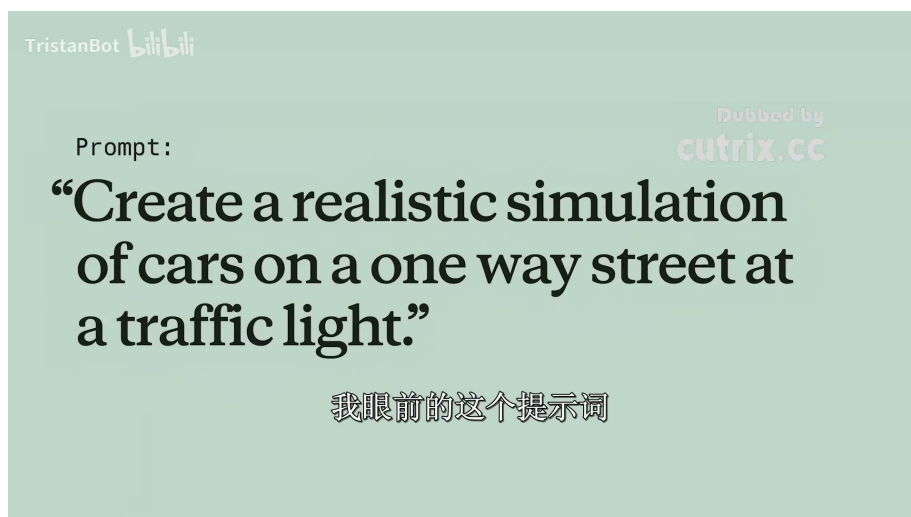


图 3：演示使用同一个提示词作为控制变量：创建一个单行道交通灯处的真实汽车行驶模拟。³

图 3 中的任务并不复杂到需要专业知识，却足够检验模型是否能处理多个约束：单行道、交通灯、车辆行驶、停车逻辑、道路布局和视觉细节。若模型只抓住“有车在路上跑”，结果就会显得粗糙；若模型同时处理交通规则、对象位置和交互行为，仿真会更接近真实场景。

为什么这个例子适合教学

交通灯仿真把质量差异变成肉眼可见的画面。读者不必理解复杂基准，也能看到 token 和时间投入增加后，车辆类型、道路组织、交通灯位置、控制面板和运动行为逐步变细。

2.2 低投入：基本结构成立，但细节较薄

00:03:10–00:03:48 是低投入运行。图像报告确认该帧显示 Claude Opus 4.7 的低投入输出：任务时间约 50.1 秒，输出 token 为 4,602。字幕也对应这一点，说它耗时大约 50 秒，生成约 4,600 个输出 token。画面里已经有道路、红灯、车辆图标和停止行为，所以它属于一个能跑起来的基线。

这一级别的特点是“足够表达题意”。字幕 61–71 里，讲者指出车辆会驶向交通灯，并在黄灯时停下，还可以调节车辆生成率和绿灯转换频率。这些都是有效功能。不过字幕 72 很关键：看起来还比较简单。这里的简单，主要体现在对象种类少、布局粗略、交通灯位置与真实道路关系还不够自然。

2.3 高投入：质量提升首先体现在约束处理

00:03:53–00:04:59 中，讲者把投入等级调高。字幕说，生成模拟的时间翻了一番，token 数量也大约增加一倍。图像报告给出更精确的画面读数：任务时间 111.3 秒，输出 token 为 9,826。质

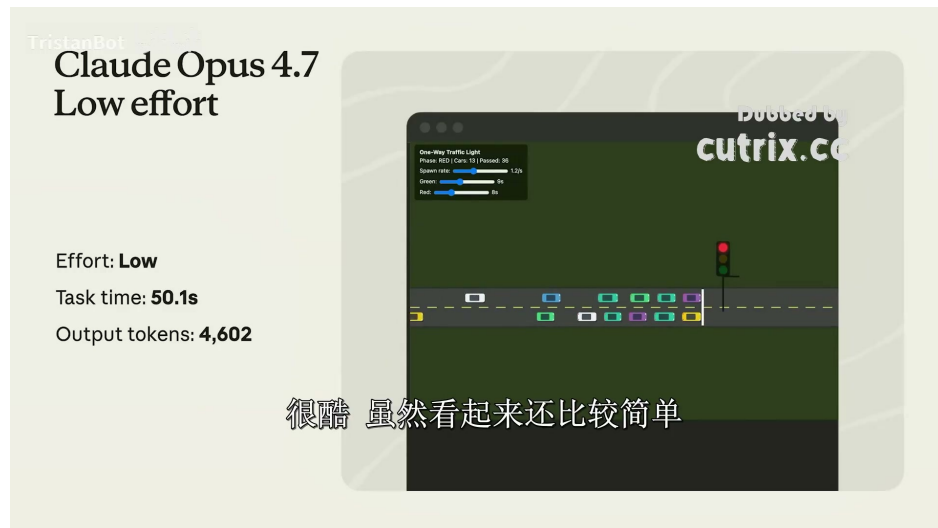


图 4：低投入输出用约 50.1 秒和 4,602 个输出 token 生成基础交通灯仿真：道路、车辆、红灯与停车行为可见，整体场景仍较简洁。⁴

量变化不只体现在“画得更满”，更体现在模型开始处理更多现实约束。

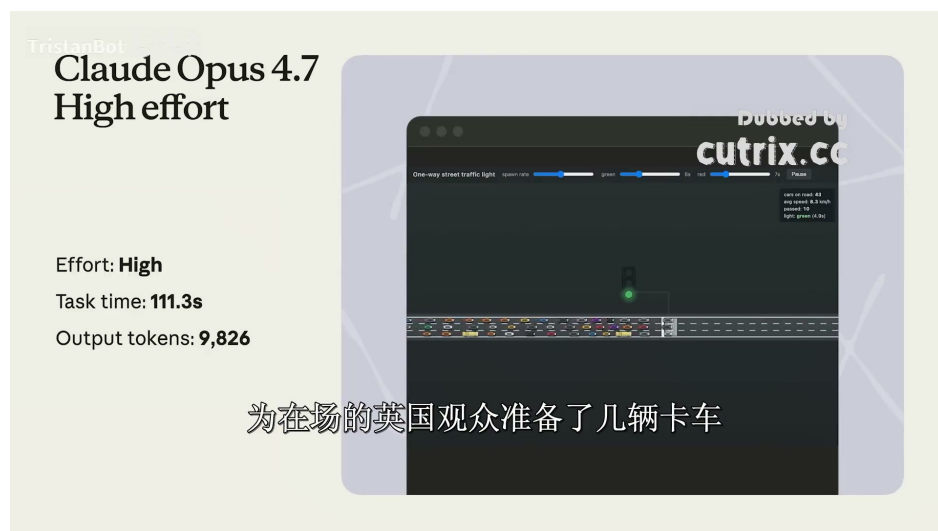


图 5：高投入输出把任务时间提高到 111.3 秒、输出 token 提高到 9,826；画面出现更丰富的车辆类型、更完整的控制栏和状态读数，道路组织也更清晰。⁵

字幕 78–97 给出了三类可观察改进。第一，车辆类型更多，画面出现不同车辆和卡车。第二，交通灯位置更合理，讲者专门比较低投入版本中交通灯在路中间的问题，并指出高投入版本似乎把灯挂到路边。第三，车辆行为更聪明，模型让司机根据其他汽车移动方式做出反应。换句话说，更多 token 没有只换来更多装饰，它也换来了对空间关系和行为规则的进一步处理。

质量提升的三层观察法

看这类演示时，可以按三层读画面：对象层看有没有车、灯、路；关系层看车道、灯位、停车线是否合理；行为层看车辆是否根据灯色和前车状态反应。低投入通常先满足对象层，高投入开始改善关系层和行为层。

讲者也没有把高投入版本说成完美。他提到交通灯虽然挂到路边，但上下颠倒，自己并不太喜欢。这句保留了一个重要边界：投入增加提高了仿真的复杂度和智能性，但不会自动消灭所有

瑕疵。

2.4 最大投入：细节密度上升，成本也突然变大

00:05:04–00:05:37 展示最大投入运行。字幕说 Claude Opus 4.7 使用了大约十倍的 token 数量和运行时间来创建模拟。图像报告确认画面读数为 593.1 秒和 52,893 个输出 token。这个跨度很大：从低投入到最大投入，等待时间从约 50 秒升到近 10 分钟，输出 token 从 4,602 升到 52,893。



图 6：最大投入输出耗时约 593.1 秒，并输出 52,893 个 token；画面加入城市背景、双车道、斑马线、交通灯状态、控制项和更多车辆图标。⁶

质量变化也非常直观。字幕 102–107 中，讲者说它更详尽，交通灯效果最好，背景天空更漂亮，汽车运动也体现出更智能的车辆行为。图 6 对应这些描述：城市背景、斑马线、两条车道、控制面板、车辆状态和灯控状态同时出现。这个结果把“思考杠杆”的收益展现得很清楚：当模型获得更多运行时空间，它能把同一个提示词展开成更完整的系统。

不过，最大投入也把成本问题暴露得很尖锐。若一个用户只需要粗略原型，近 10 分钟和 5 万多个输出 token 可能过重；若任务目标是展示高质量仿真，额外成本才可能合理。这里没有单一答案，只有质量线与资源边界的匹配。

不要只看最终画面

最大投入的画面最丰富，但它也消耗最多时间和 token。工程决策不能只问“哪一个最好看”，还要问“这一级质量是否值得 593.1 秒和 52,893 个输出 token”。忽略成本，思考杠杆会变成资源黑洞。

2.5 一个抽象公式：投入、质量、时间一起移动

交通灯演示可以压缩成一个抽象关系。它只描述本章现象，不代表视频给出了精确数学定律：

$$E \uparrow \Rightarrow B_{\text{tok}}, T \uparrow, \quad Q \text{ 通常随之改善}$$

- E ：努力等级，本章用低投入、高投入和最大投入三个演示档位展示它的效果。
- B_{tok} ：本次运行实际占用的 token 空间，图中输出 token 分别约为 4,602、9,826、52,893。

- T : 任务耗时，图中分别约为 50.1 秒、111.3 秒、593.1 秒。
- Q : 演示质量，表现为仿真是否更真实、更合理、更符合提示词约束。

这个式子的重点在后半句“通常”。低到高再到最大，本次演示的质量明显上升；换成其他任务时，收益曲线可能更早变平。读者应当把这章当作可视化样例，避免把最大投入当成默认策略。后面的章节会继续讨论怎样用评估选择合适档位。

2.6 从几分钟仿真到长时程自主工作

00:05:38–00:06:27 中，讲者把话题从交通灯扩展到更长时程的模型工作。他先总结：花费的 token 越多，花费的时间越长。随后指出，Claude 的工作量可能从几秒、几分钟或几小时，逐渐增加到几天、几周甚至几个月。这里的“工作量”可以理解为模型可持续推进任务的时间跨度，而不只是一次回答的长度。

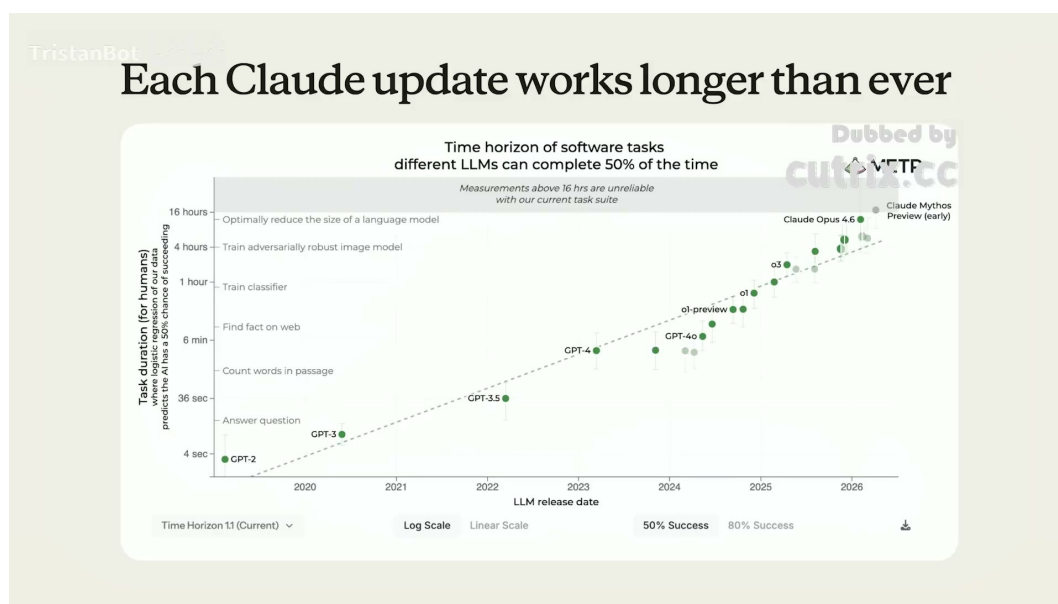


图 7: METR 时间跨度图展示后续 Claude 更新覆盖更长软件任务时长的趋势；图中标注 50% 成功率口径，并提示当前任务集在 16 小时以上的测量可靠性不足。⁷

图 7 将这件事放到软件任务时间跨度上。图像报告确认它是 METR 品牌的长时程图，纵轴是软件任务时长，横轴是模型发布日期，并包含 50% 成功率口径以及“16 小时以上测量可靠性不足”的限制说明。字幕 124–126 也提到，最新模型之一在大约 16 小时的人类工作量下达到 50% 准确率。

这张图不能被过度解读成对未来的精确预言。更稳妥的读法是：模型更新、训练时计算和测试时计算结合后，模型能覆盖的任务时长正在扩大。交通灯演示讲的是一次提示词内的投入差异；METR 图讲的是模型能承担的工作跨度正在向更长任务移动。二者共同指向同一个主题：运行时资源分配会变得越来越重要。

2.7 本章小结

本章覆盖 00:02:39–00:06:27，用交通灯仿真把“思考杠杆”变成可观察实验。同一个提示词在低投入、高投入和最大投入下分别消耗约 4,602、9,826、52,893 个输出 token，任务时间约为 50.1

秒、111.3 秒、593.1 秒。随着投入增加，仿真从基础道路和红灯，发展到更丰富的车辆类型、更合理的灯位、更智能的车辆行为，再到带城市背景、斑马线、控制项和更多状态信息的完整场景。

这章也把代价讲明白了：更高投入会带来更长等待和更多 token 消耗。最大投入最能展示能力上限，却未必适合每个任务。最后的 METR 图把视野推向长时程自主工作：后续模型越来越可能处理以小时计的软件任务，但图中也明确提示 16 小时以上测量仍有可靠性限制。到这里为止，读者需要记住的判断很朴素：投入强度确实能改变结果，但它必须和任务质量线、成本与等待时间一起评估。

3 测试时计算的三种载体

3.1 把“多想一会儿”拆成可管理的资源

00:06:28–00:08:43 这一段把前两章里的现象落到工程接口上。前面已经看到，同一个交通灯仿真任务，Claude 在更高努力程度下会消耗更多令牌和更长时间，也会生成更复杂的行为。这里要回答一个更基础的问题：这些测试时计算到底花在了哪里？

演讲者给出的拆法很直接：测试时计算可以表现为任何花费令牌的形式，通常发生在推理阶段。为了便于控制和解释，可以把它分成三种载体：思考、工具使用、最终文本。三者像一张工作台上的三个区域：草稿纸负责推演，工具负责触达外部世界，最终文本负责把结果交给用户。

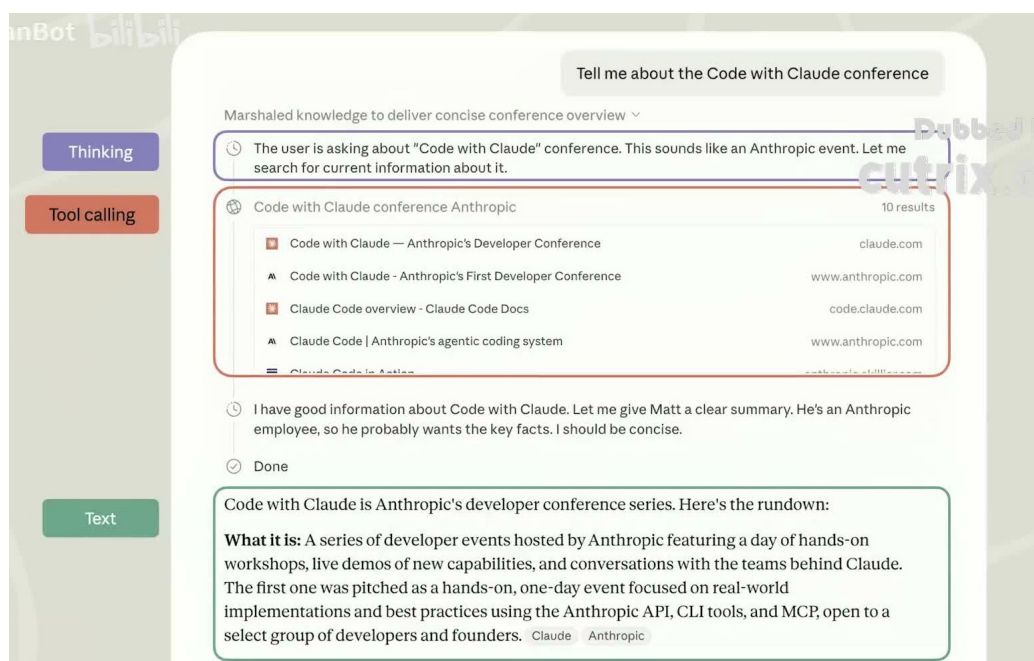


图 8：Claude UI 示例把测试时计算拆成三个可观察的运行通道：左侧标签依次标出 Thinking、Tool calling 与 Text；中间内容展示一次搜索工具调用，以及随后生成的最终文本。它适合用来说明“思考”是内部草稿空间，“工具调用”是外部世界接口，“最终文本”是交付给用户的回答。⁸

三种运行时载体

思考是 Claude 的内部推理空间，可以理解为草稿纸：它在这里整理问题、读取提示中可用的数据、规划后续步骤。

工具使用是 Claude 与外部世界交互的接口：搜索网页、调用服务、读取或写入文件、访问业

务系统，都属于这一类。

最终文本是 Claude 对用户可见的回答、总结、追问或下一步说明。

这个拆分的价值在于，它把一个容易被说得很玄的概念变成了可检查的资源流。模型花费的令牌，有些用于中间推理，有些用于发起和理解工具调用，有些用于输出给用户。等待时间也按同样的路径积累：模型思考要时间，工具返回要时间，最终文本生成也要时间。

3.2 思考：模型的草稿纸和推理空间

字幕块 130–135 把第一种载体称为“思考”，并解释为 Claude 的推理空间。这里的“思考”不应该被理解成神秘的意识活动；在工程语境中，它更像一块临时草稿纸。模型把用户问题、上下文、约束条件和可用数据放到这块草稿纸上，尝试组织出一条可执行的解决路径。

一个人做复杂题时，通常会先列条件、拆步骤、排除错误路线。模型的思考空间承担类似作用。它让 Claude 可以在立即给出答案之前，先处理几个关键问题：当前任务真正要求什么？提示里有哪些数据能用？是否需要搜索、读取文件或调用服务？答案是否需要先验证再输出？

“草稿纸”类比的边界

草稿纸类比帮助读者理解思考空间的作用，但它无法覆盖模型内部机制的全部细节。真正的语言模型仍然通过令牌序列进行计算；所谓草稿纸，强调的是中间推理文本为后续步骤提供了可复用的结构。用数学化的简写说，可以把一次回答看成

$$y_{\text{final}} = g(x, r, u),$$

其中 x 是用户输入与上下文， r 是中间推理轨迹， u 是工具返回或外部观察， y_{final} 是最终输出。这个式子只是教学抽象，视频并未把它作为测量定律给出。

- x ：用户问题、系统约束、已有上下文和提示中的数据。
- r ：模型在推理空间中形成的中间步骤、假设、计划和检查。
- u ：工具调用后获得的外部信息，例如搜索结果、文件内容或服务返回。
- y_{final} ：用户最终看到的文本回答或追问。

在后续章节里，“是否开启思考”会被进一步讨论。这里先抓住一个更底层的事实：思考本身会消耗令牌和时间。它能提高复杂任务的处理质量，也会推高直接成本。

3.3 工具使用：连接外部世界的接口

第二种载体是工具调用。字幕块 136–147 给出的例子是网络搜索：用户要求 Claude 搜索更多关于 Anthropic 开发者大会的信息。演讲者随后把范围扩大：工具可以是很多东西，可以调用销售部门的服务，也可以直接写入文件系统。

工具使用的关键作用，是把模型从“只处理上下文里的文字”扩展到“能观察和改变外部状态”。搜索让模型获得新资料，文件读写让模型参与本地工作流，业务服务调用让模型进入真实系统。对开发者来说，这一层最像 API 边界：Claude 发出调用，外部系统返回结果，模型再把结果纳入后续推理。

工具调用增加能力，也增加风险面

工具让 Claude 能处理真实任务，同时也引入权限、延迟、失败重试和状态改变等问题。读文件和写文件的风险不同，搜索资料和调用业务服务的后果也不同。设计工具接口时，不能只看模型是否“会用”，还要看它被允许访问什么、调用结果怎样验证、失败时怎样收敛。

工具调用和思考之间还有一个重要关系：工具返回的信息常常会改变后续推理。若搜索结果与初始假设冲突，模型应当修正计划；若文件内容显示需求边界不同，模型应当重新组织答案。第四章的交错思考正是围绕这个关系展开。

3.4 最终文本：把工作结果交给用户

第三种载体是最终文本。字幕块 147–148 说明，它代表 Claude 对问题的总结或进一步询问。最终文本看起来只是“回答”，实际承担两种不同任务：一种是交付结论，另一种是澄清下一步。

交付结论时，最终文本要把前面的推理和工具结果压缩成用户可用的形式。澄清下一步时，最终文本要指出缺失信息、风险或需要用户决策的地方。复杂工作里，这两者经常交替出现：模型先给出部分发现，再问一个关键问题，然后继续调用工具或推理。

这解释了为什么只盯着最终输出的字数，会低估真实成本。用户看到的文本可能很短，背后却经历了长时间思考和多轮工具调用；也可能最终文本很长，但中间推理很少。成本核算要看三种载体的总和。

3.5 努力程度与预算：软旋钮和硬边界

00:07:30 之后，演讲者转向成本控制。测试时计算有直接成本：令牌计数和所花时间。用户自然会希望对 Claude 每天实际做的事有更多控制，于是出现两类控制方式：努力程度和预算。

努力程度与预算的定义

努力程度是一种软旋钮，通常表现为从低到高的等级。分配更高努力程度时，Claude 往往工作更久，消耗更多令牌，并尝试做更充分的推理。

预算是一种更严格的约束机制，可以是最大令牌限制，也可以是任务预算。它的作用是给模型划出资源边界，避免投入越过上限。

两者的差别可以用驾驶类比理解。努力程度像油门深浅：踩得更深，车更愿意加速，油耗也更高。预算像油箱容量或限速牌：无论驾驶员多想继续，边界到了就必须收束。实际系统常常需要同时设置两者：努力程度决定 Claude 倾向于投入多少，预算决定投入不能越过哪条线。

可以把本章的成本关系写成一个简化公式：

$$Q \approx f(M, E, B_{\text{tok}}, \text{task}, \text{context}), \quad C, T \uparrow \text{ as } E \text{ and generated tokens increase.}$$

- Q : 输出质量或任务成功率。
- M : 所选模型或模型家族。
- E : 努力程度，例如低、中、高、超高、最大。
- B_{tok} : 令牌预算或最大令牌上限。
- task : 任务本身的难度、风险和结构。

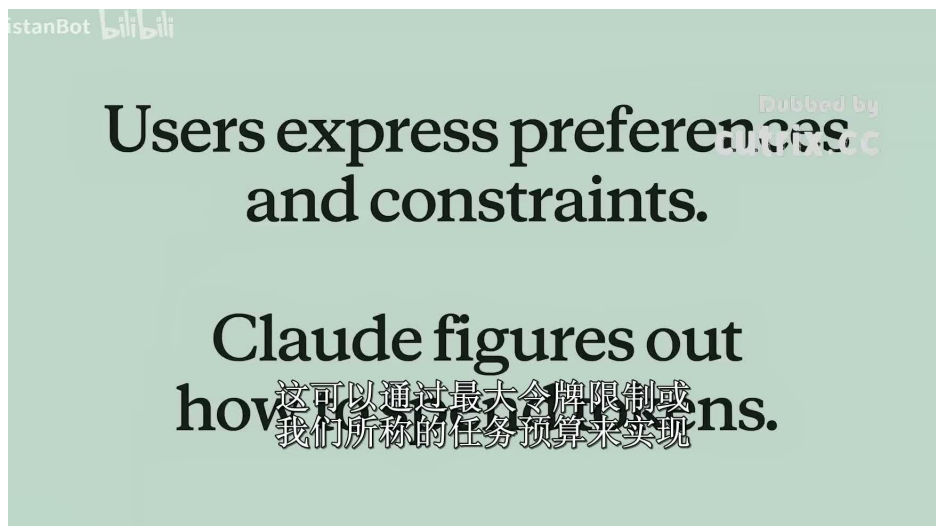


图 9: 控制面总结页给出本节的产品抽象: 用户表达偏好与约束, Claude 再决定如何花费 token。这里的“偏好”对应努力程度这类软控制, “约束”对应最大 token 限制或任务预算这类硬边界。⁹

- **context**: 提示、文件、工具返回和用户提供的背景。
- C : 运行成本，常随令牌数和工具使用增加。
- T : 端到端耗时，常随思考、工具等待和输出长度增加。

这个公式是讲义中的抽象地图。视频支持的是定性关系：更高努力程度通常带来更多金牌和更长时间，质量可能提升；具体提升多少，要靠后续章节里的评估方法确定。

3.6 本章小结

本章把 00:06:28–00:08:43 的内容整理成一个运行时资源框架。测试时计算可以落在三种载体上：思考、工具使用、最终文本。思考是 Claude 的推理空间，帮助它组织问题和后续步骤；工具使用是外部世界接口，让它能搜索、调用服务、读写系统；最终文本是交付给用户的总结、答案或追问。

成本也因此有了更清晰的来源。令牌和时间会花在最终回答上，也会花在中间推理和工具交互上。努力程度是控制投入倾向的软旋钮，预算是限制资源上限的硬边界。理解这一区分后，第四章才能自然进入下一步：模型究竟应当先思考再行动，还是在行动和思考之间反复切换，并由自己判断何时需要思考。

4 从扩展思考到交错思考,再到自适应思考

4.1 理想状态：模型自己判断该投入多少

00:08:47-00:12:02 的主线，是从固定分配的思考走向更灵活的运行时控制。上一章已经定义了努力程度和预算：努力程度像软旋钮，预算像硬边界。这里要回答另一个问题：思考发生在什么时候，顺序由谁决定？

字幕块 176-179 给出理想状态：用户问一个问题，Claude 能知道自己应当投入多少努力。人类用户仍然希望保留一个可以随时调整努力程度的杠杆，因为不同任务的风险、时间要求和质量

标准差异很大。可操作的系统需要同时满足两点：用户能给约束，模型也能按任务动态安排工作。

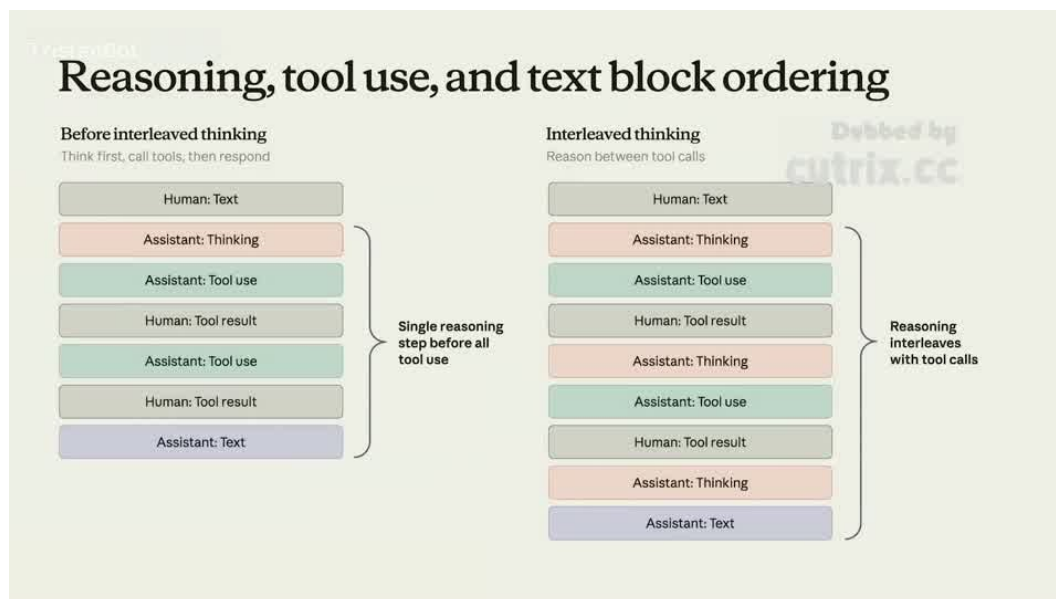


图 10：时间线对比了两种块顺序：左侧是在所有工具调用前集中做一次思考，右侧是在工具调用之间插入思考步骤。图中粉色块表示 Assistant: Thinking，绿色块表示 Assistant: Tool use，灰色块表示 Human: Tool result，紫色块表示 Assistant: Text。¹⁰

本章三个术语

扩展思考指早期或固定式的思考模式：模型先根据分配到的思考令牌集中思考一段时间，再执行工具调用，最后生成回答。

交错思考指行动和思考交替出现：做一点事，思考一下，再做一点事，再思考一下，最后形成结果。

自适应思考指模型根据任务、上下文和自身能力，选择何时思考、为何思考、是否调用工具以及何时输出文本。

三者的差异并不在于“有没有思考”这个开关，而在于控制权的位置。扩展思考把思考放在前置块里；交错思考允许工具调用之后再重新推理；自适应思考进一步让 Claude 判断每一步是否值得花费思考令牌。

4.2 扩展思考：先集中推理，再执行工具

字幕块 180–186 描述了初始状态：用户提问后，Claude 根据分配的思考令牌数思考一段时间，然后执行一系列工具调用，读取每个结果，直到输出形成，最后用户得到回答。

这个模式可以写成一条线性流程：

问题 → 思考块 → 工具调用序列 → 最终回答。

- 问题：用户输入和上下文。
- 思考块：预先分配的一段推理空间。
- 工具调用序列：搜索、读取、调用服务等外部动作。
- 最终回答：汇总工具结果后输出给用户的文本。

扩展思考的好处是简单，系统容易实现，用户也容易理解：先给模型一块思考预算，让它认真规划，再让它执行。问题也很清楚：工具返回之后，环境可能已经提供了新证据，早先的推理计划可能需要修正。如果流程只允许前面集中思考一次，后面的行动就容易带着过期假设继续走。

固定前置思考的主要风险

复杂任务里的关键信息常常来自工具返回，初始提示只提供起点。若模型在看到工具结果之前已经把计划定死，就可能出现三类问题：搜索后没有重新判断证据质量，读取文件后没有修正任务边界，调用服务后没有根据失败或异常返回调整下一步。

演讲者用人类工作方式做类比：别人问一个问题时，人通常不会先静止地处理完整个方案，然后突然执行一堆步骤并给出答案。更自然的方式是在做事过程中不断检查和调整。

4.3 交错思考：工具之后还要重新推理

字幕块 193–199 给出交错思考的动机和定义：做点事，思考一下，再做点事，再思考一下，最后得到结果。它让 Claude 能在每次工具调用后进行思考。

可以把交错思考看成一条带反馈的流程：

$$\text{问题} \rightarrow r_1 \rightarrow u_1 \rightarrow r_2 \rightarrow u_2 \rightarrow \cdots \rightarrow y_{\text{final}}.$$

- r_i ：第 i 次思考，负责根据当前信息更新计划。
- u_i ：第 i 次工具调用或外部观察。
- y_{final} ：最终文本输出。

这个结构的关键是“每次新观察之后，都允许重新组织推理”。例如，搜索结果可能显示原先关键词不准确；读取文件可能发现用户给出的错误线索；运行测试可能暴露新的失败点。交错思考让模型在每个观察点上停一下，把新证据纳入下一步行动。

交错思考像调试循环

软件调试很少是一口气写完整个修复方案。工程师通常先读日志，形成假设，改一处代码，跑测试，再根据失败信息调整。交错思考把这种“观察–假设–行动–再观察”的循环放进模型运行时，使工具结果能真正改变后续路线。

交错思考已经比固定前置思考更贴近真实工作，但它仍然带着一个隐含设定：每次工具调用后都倾向于思考一下。对困难任务，这很合理；对简单任务，频繁思考可能只是额外成本。

4.4 自适应思考：把何时思考交给模型判断

字幕块 201–207 把自适应思考定义为交错思考的演进：给模型更多关于何时思考、为何思考的控制权。Claude 可以根据手头问题选择调用工具、输出文本，或者按它习惯的逻辑顺序进行思考。

这一步的核心变化，是模型获得了“是否值得思考”的局部判断权。自适应思考是一种运行时控制方式，也不要求预先把请求分类。字幕块 229–234 说得很具体：系统告诉 Claude，它有思考工具，有思考能力，随时都能使用。接下来，模型在任务过程中决定是否调用这项能力。

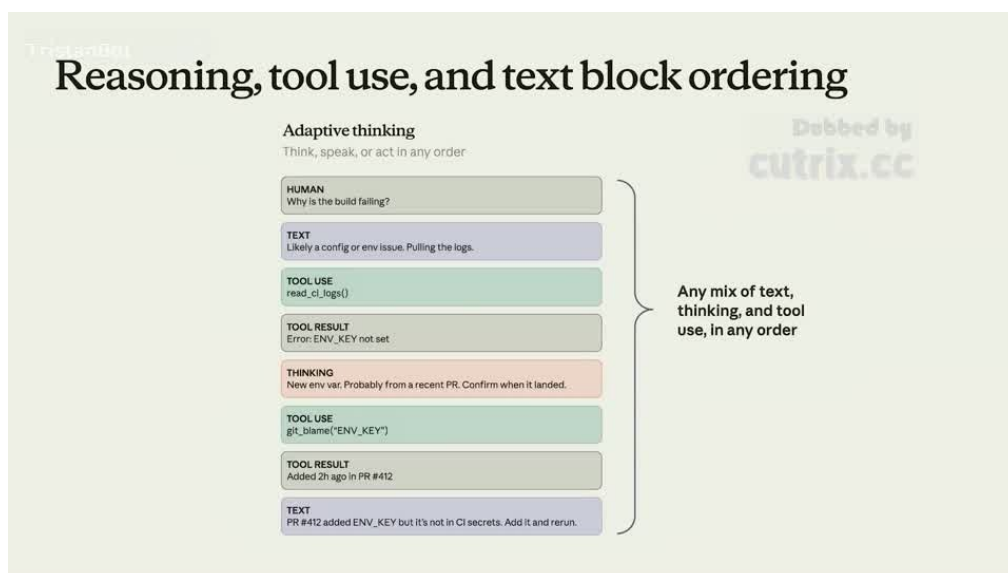


图 11: Adaptive thinking 的图示把控制权进一步交给模型：它可以按任务需要混合文本、工具使用、工具结果和思考步骤。右侧括号明确写出 ‘Any mix of text, thinking, and tool use, in any order’，这正是 “何时思考、为何思考、是否思考” 由模型结合上下文决定的视觉证据。¹¹

自适应思考的定义

自适应思考是一种运行时控制方式：Claude 在每一步根据任务难度、模型能力、上下文信息和外部观察，决定是否思考、思考多久、是否调用工具、是否直接输出文本。它关注的是 “何时使用思考能力”，重点不在给模型贴固定标签。

字幕块 219–228 给了一个直观例子。若问题是 $10 + 10$ ，人会立即回答 20；若问题是一组极其困难的博士级习题，人可能会深思，而且不同人需要的时间不同。任务、所用模型和上下文共同决定了需要多少思考。

这个类比可以进一步压缩成一个决策式：

$$a_t \in \{\text{think, use tool, write}\} \quad \text{chosen from } (\text{task}, M, \text{context}_t, B_{\text{task}}).$$

- a_t : 第 t 步的动作。
- think: 使用思考能力，消耗推理令牌。
- use tool: 调用外部工具，获得新观察或改变外部状态。
- write: 生成最终文本或向用户追问。
- task: 任务难度、风险和目标。
- M : 当前模型的能力水平。
- context_t : 第 t 步已经拥有的上下文和工具返回。
- B_{task} : 任务级预算边界，可包含令牌、时间、工具和复核约束。

这只是帮助理解的抽象表达，视频没有把它作为精确算法提出。它说明自适应思考的焦点在动作选择：简单步骤可以直接写，证据不足时可以调用工具，推理复杂时可以使用思考空间。

4.5 效率：为什么自适应思考可能更接近帕累托最优

字幕块 242–246 提到，屏幕上展示的基准测试基于自适应思考，结果发现相比以往交错思考模式，这种方式在效率上是帕累托最优的。这里的“帕累托最优”需要谨慎理解：在质量、成本、时间之间，若一种方案能在不牺牲某个维度的情况下改善另一个维度，它就支配了旧方案；若再也找不到这种单向改善，就处在帕累托前沿上。

	Opus 4.7	Opus 4.6	Mythos Preview
Agentic coding SWE-bench Pro	64.3%	53.4%	77.8%
Agentic coding SWE-bench Verified	87.6%	80.8%	93.9%
Agentic terminal coding Terminal-Bench 2.0	69.4%	65.4%	82.0%
Multidisciplinary reasoning Humanity's Last Exam	46.9% no tools	40.0% no tools	56.8% no tools
	54.7% with tools	53.3% with tools	64.7% with tools
Agentic search BrowseComp	79.3%	83.7%	86.9%
Scaled tool use MCP-Atlas	77.3%	75.8%	—
OSWorld-Verified			
Agentic financial analysis	64.4%	60.1%	—

图 12: 表格把 Opus 4.7、Opus 4.6 与 Mythos Preview 放在同一组任务上比较，包括 agentic coding、terminal coding、multidisciplinary reasoning、agentic search、scaled tool use 与 financial analysis。讲者用它支持自适应思考在效率上的 Pareto 改进主张；底部有字幕遮挡，正文引用时应聚焦表头、可读行与整体比较关系。¹²

帕累托前沿的直觉

假设横轴是成本 C ，纵轴是质量 Q 。一个配置若成本更低且质量不差，或质量更高且成本不高，就比另一个配置更优。帕累托前沿上的点没有被其他点完全压过。视频中的说法可以理解为：自适应思考让模型减少不必要的思考，同时在需要深推理的任务上保留能力，因此更容易接近质量和成本的有效边界。

自适应思考的效率来源很朴素。简单问题无需强行花费思考令牌；工具返回已经足够明确时，可以直接总结；发现证据冲突或任务难度上升时，再投入更多推理。它把“每一步都想一想”的固定习惯，改成“值得想时才想”的资源分配。

这也为第五章埋下伏笔。若用户把思考理解成一个开关，就容易在“开”和“关”之间摇摆。更准确的操作面是能力加约束：让 Claude 拥有思考能力，同时用努力程度、预算、工具权限和任务目标限制它怎样花费资源。

4.6 本章小结

本章梳理了 00:08:47–00:12:02 的三阶段演进。扩展思考把思考放在前置块中：先集中推理，再执行工具，再输出答案。交错思考把推理插入工具调用之后：行动、观察、思考、再行动，使工具返回能改变后续路线。自适应思考进一步把何时思考、为何思考的控制权交给 Claude，让它根据任务、模型和上下文决定下一步是思考、调用工具，还是输出文本。

这条演进线的核心，是让思考出现在值得花费资源的位置，避免把“永远更多的思考”当成目标。简单算术可以直接回答，博士级难题需要深推理，工具返回冲突时需要重新判断。自适应思考的目标，是在质量、时间和成本之间更有效地分配测试时计算。

5 思考开关的误区：把能力留给模型

5.1 把思考当开关，会丢失控制精度

00:12:02–00:13:33 这一段处理一个很常见的产品误解：用户容易把“思考”理解成等待时间的开关。想要更好的答案，就打开思考；想要更快的回答，就关掉思考。这个直觉有现实基础，因为前几章已经看到，更高努力等级通常会带来更多 token 和更长等待时间。

问题在于，开关只能表达允许或禁止，无法表达“希望 Claude 投入多深”。字幕块 247–255 说得很清楚：开启或关闭思考开关时，用户没有传达所需的思考深度；关闭时还会关掉一项核心能力。更准确的控制面应当包含两个部分：能力保持可用，资源投入由努力等级、预算和任务约束调节。



图 13：这一页把控制方式从二值开关改写成努力程度旋钮：用户需要调节投入深度，同时保留思考能力本身。¹³

本节的核心判断

思考是一项能力，努力等级是投入强度，预算资源边界。把思考能力关掉，相当于让 Claude 少了一种处理复杂任务的手段；设定努力等级和预算，才是在保留能力的同时控制成本。

这一区分会直接影响系统设计。如果产品只提供“思考开/关”，用户就被迫在质量和速度之

间做粗糙选择。如果产品提供努力等级、最大 token、时间预算和工具权限，用户能表达更具体的需求：这个任务可以慢一点，但不要超过某个 token 上限；这个任务很简单，需要低延迟；这个任务需要搜索和推理，但工具调用必须受限。

5.2 工具类比：让能力可用，再让模型判断何时使用

字幕块 262–265 给出一个更容易理解的类比：搜索工具。用户通常不会要求 Claude 永久联网，也不会要求它每次都联网。更合理的方式是把搜索工具提供给模型，让它根据任务判断何时搜索。搜索能力存在，并不代表每个回答都要搜索；思考能力存在，也不代表每一步都要长时间推理。

这个类比把“思考”从神秘概念拉回到工具箱逻辑。一个工程师的工具箱里有搜索、调试器、日志、测试命令和草稿纸。工程师不会每次都打开所有工具，也不应被禁止使用关键工具。正确做法是给出任务目标、边界和验收标准，让执行者在过程中选择合适工具。

能力可用与能力滥用的区别

能力可用表示 Claude 可以在需要使用思考空间、工具调用或最终文本。能力滥用表示它在简单任务上消耗过多 token、调用多余工具、产生不必要等待。努力等级和预算的价值，正是把“可用”与“滥用”分开。

如果把搜索工具强行关掉，模型在需要外部事实时只能猜测。如果把思考能力强行关掉，模型在需要多步推理时只能快速输出。两种做法都会让复杂任务的失败形态更隐蔽：答案看起来流畅，内部检查却不足；流程看起来快速，后续人工返工增加。

5.3 同事类比：给约束，让对方完成分析

字幕块 266–276 转向同事协作。演讲者的意思很朴素：和一个同事协作时，通常不会要求对方不用心分析，只给一个简单答案。更自然的做法是告诉对方约束条件、背景知识和任务目标，让对方基于自己的经验执行，再带着答案回来。

把 Claude 当成可协作的执行者时，提示词也应承担类似作用。用户需要提供：

- **目标：**这次任务要解决什么问题，输出要服务谁。
- **约束：**时间、成本、工具权限、格式、事实来源和风险边界。
- **上下文：**已有资料、已知假设、失败案例、相关文件或业务背景。
- **验收线：**什么算完成，什么情况需要追问，什么错误不能接受。

这些信息比一个单独开关更有表达力。开关只说“允许思考吗”；约束和验收线会告诉模型“在什么范围内把任务做好”。对于开发者，这意味着 API 或产品界面应当逐步走向结构化控制：努力等级、token 上限、工具白名单、任务预算、输出质量标准、必要时的人类复核点。

开关式控制的失败模式

开关式控制容易制造两种失败。第一种是过度节省：复杂任务被迫快速输出，模型少做关键检查。第二种是过度消耗：简单任务也进入长推理路径，延迟和 token 成本上升。两种失败都源于控制信号太粗，无法表达任务差异。

5.4 为下一步做准备：从能力开关走向评估校准

本节不解决“某个任务该选哪个努力等级”，这个问题由后续评估和经验法则处理。这里先确定控制原则：保留能力，明确约束，用任务质量线决定投入。这个原则解释了为什么第六章要引入评估集。只靠直觉很难判断一个任务需要多少思考，真实困难样本才能暴露低努力配置会漏掉什么，也能暴露高努力配置是否已经进入收益平坦区。

可以把本节关系写成一个简单判断式：

$$\text{Control} = \text{Capability available} + \text{Constraints} + \text{Evaluation feedback}.$$

- **Capability available:** 思考、工具、文本输出这些能力保持可用。
- **Constraints:** 努力等级、token 上限、时间窗口、工具权限和任务目标。
- **Evaluation feedback:** 真实任务上的质量、速度和成本反馈。

这个式子只是帮助理解的抽象。它想表达的重点是：控制 Claude 的方式，应当像管理一个可协作系统。能力保持可用，资源受约束，表现由评估校准。

5.5 本章小结

本章覆盖 00:12:02–00:13:33。演讲者批评的核心点，是把思考当成二值开关会损失控制精度。开启或关闭思考，无法表达用户希望 Claude 投入多深；关闭思考还会移除一项处理复杂任务的核心能力。

更稳的设计方式，是把思考看成能力，把努力等级看成投入旋钮，把预算看成边界。搜索工具类比说明：能力存在，并不代表每次都要使用；同事协作类比说明：用户应提供目标、约束和背景，让 Claude 在边界内判断怎样完成工作。下一章将用评估集和努力等级扫描，把这套控制原则落到可测量的选择流程里。

6 用评估校准努力等级

6.1 真实难题才会暴露投入差异

00:13:33–00:16:07 这一段把前面的控制原则变成操作方法。演讲者先回到努力程度曲线：随着努力程度提升，模型能力也会上升。接着他给出最关键的建议：建立一个高质量问题测试集，包含真正希望 Claude 解决的难题，并在不同努力水平下评估模型表现。

这一步看似普通，实际决定了团队能否控制成本。如果评估集太简单，低努力配置会显得很划算，因为所有配置都能过线；上线后遇到长链推理、工具调用、多约束规划时，问题才会暴露。评估集应当覆盖那些真正让团队头疼的样本：难复现的 bug、边界条件多的总结、需要多步验证的研究问题、容易漏掉约束的代码任务。

评估校准的最小流程

先收集真实困难任务，再按多个努力等级运行同一组样本，记录质量、token、耗时和失败类型。选择初始努力等级时，优先选刚好稳定达到质量线的最低资源档位。

这里的“刚好”很重要。若质量线是 $Q_{\min}$ ，团队寻找的是满足

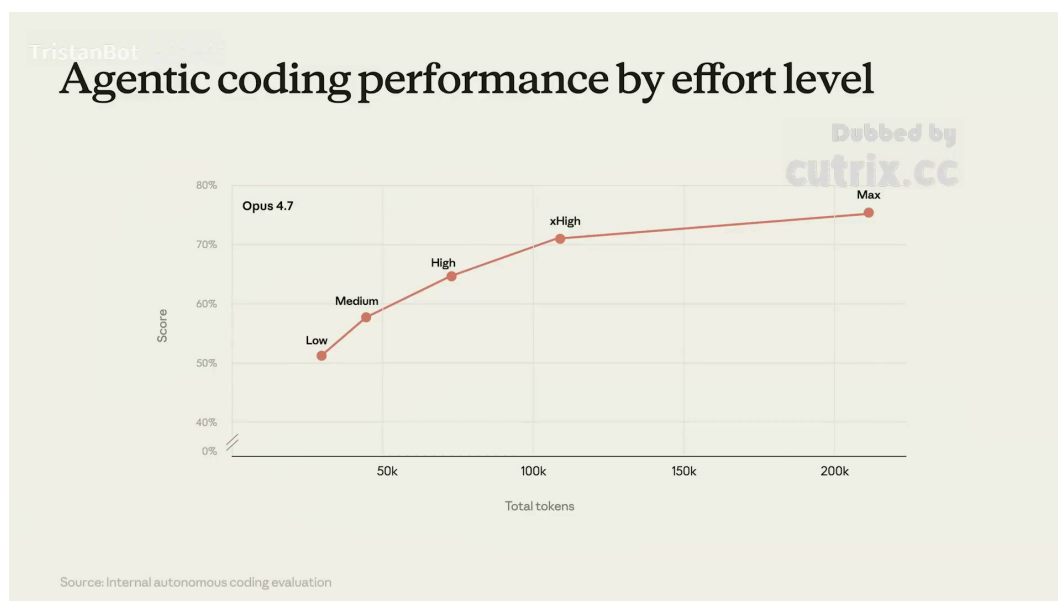


图 14: 努力程度曲线展示了 Low、Medium、High、xHigh、Max 等档位的性能变化。曲线在高档位附近趋于平缓，说明更高投入可能继续改善质量，但边际收益会下降。¹⁴

$$Q(E) \geq Q_{\min}$$

的最低努力等级 E ，同时观察成本 $C(E)$ 和耗时 $T(E)$ 。这个表达是评估决策的简写，视频没有把它作为测量公式给出。它提醒团队把“更强”翻译成可比较的指标：通过率、事实召回、错误率、人工返工时间、平均 token 和端到端延迟。

6.2 边际收益递减：最高档经常很贵

字幕块 290–297 点出图表最值得注意的地方：不同任务中，提升努力程度带来的边际效益会递减。在示例里，最高等级的 token 消耗接近超高级别的两倍，性能提升却很小。这个现象对产品配置很关键，因为它说明“更高档”并不自动等于“更合理”。

不要把最高档当作默认答案

最高努力等级适合极难、高价值、允许长等待的任务。若任务已经在 xHigh 或 High 达到质量线，继续升到 Max 可能只是在平坦区间上增加成本。评估的意义，就是找出这条平坦区间从哪里开始。

边际收益递减可以用日常学习类比。一个人准备考试，前两小时复习可能补上大量知识漏洞；第十小时继续复习，改善幅度往往小很多。模型努力等级也类似：从低到中、高，模型得到更多检查和搜索空间；接近最高档时，额外 token 可能只是在已经较完整的答案上做细小修补。

在工程实践里，团队应当记录三类曲线：

- **质量曲线：** 不同努力等级下的成功率、评分或人工验收结果。
- **成本曲线：** 平均 token、工具调用次数、费用。
- **延迟曲线：** 平均耗时、P95 耗时、超时比例。

只有把三条曲线放在一起看，才知道某个档位是合理投入，还是质量提升已经追不上成本上升。

6.3 低努力的价值：速度约束会改变搜索路径

字幕块 298–303 讨论低努力等级。低努力能处理许多任务，代价是牺牲一部分智能，以换取速度。讲者把它描述为一种“受非智能约束”的任务状态：模型不能过度深思，因此会倾向于更快、更短、更直接的策略。

这并不意味着低努力只会变差。对于分类、抽取、格式化、简单摘要这类任务，低努力常常足够。更有意思的是，低努力有时还会改变搜索路径，让模型找到看似偷懒却有效的策略。接下来的 Pokemon Red 例子正是为了说明这一点。

6.4 Pokemon Red 例子：低努力带来的捷径策略

00:15:01–00:16:07 中，讲者提到一个他很喜欢的评价方式：把 Claude 放进 Pokemon Red，给它游戏画面和类似手柄按键的工具，让它尝试挑战四天王。这个评估很适合说明工具型任务里的努力约束，因为模型既要看看画面，又要规划路线，还要通过工具执行动作。



图 15: Pokemon Red 评估画面显示，模型接收游戏状态，并可通过控制器式工具采取行动。这个设置把“看见状态、规划路线、执行动作”放在同一任务中。¹⁵

字幕块 315–330 描述了低努力模式下的有趣行为：Claude 会使用 Repel、药水、Escape Rope 等方式减少往返和遭遇战，在草丛遇敌时迅速逃跑，甚至形成一种更快完成导航的路线。讲者把它形容为“懒洋洋”，这里更严谨的说法是：资源约束改变了策略偏好。模型减少长时间规划，转而寻找能快速推进的局部动作。

约束会塑造策略

低努力模式像给模型设定较短的思考窗口。窗口变短后，模型可能偏向立即可执行、反馈快、路径短的动作。这样的策略在某些环境中会变成捷径，在另一些环境中会造成短视。评估的作用，是判断这种约束到底提高了效率，还是伤害了任务目标。

这个例子不能被读成“低努力更聪明”。它说明的是另一层机制：不同努力等级改变模型搜索解法的方式。高努力可能做更全面的规划，低努力可能压缩路径并减少犹豫。哪一种更好，取决于

任务目标、反馈密度和失败代价。游戏导航中快速逃跑可能有效；法律分析、代码迁移、医学建议这类任务中，快速跳过细节会带来严重风险。

6.5 怎样建立可用的努力等级评估

把本节内容落到团队实践，可以得到一个小型评估方案：

1. **收集样本**：从真实失败、人工高成本任务、用户投诉、复杂工单中选样本。
2. **划分任务类型**：把样本按摘要、代码、检索、规划、工具操作、分类等类型分组。
3. **定义质量线**：每组任务写清验收标准，例如通过测试、事实完整、格式正确、风险点召回。
4. **扫描努力等级**：同一模型下跑 Low、Medium、High、xHigh、Max，记录质量、token、耗时。
5. **看失败类型**：区分事实遗漏、推理不足、工具调用错误、输出冗余、延迟过高。
6. **选择默认值**：选达到质量线且资源最低的档位，再为高风险样本保留升级路径。

评估集的作用

评估集的作用在于把任务质量线、成本曲线和失败类型放到同一张表里，让团队知道什么时候可以省 token，什么时候必须给 Claude 更多推理空间。它服务于配置校准，不追求证明某个档位永远最好。

这一章的边界也要明确：它只讨论努力等级如何校准。模型大小与努力等级怎样组合，留到下一章处理。团队可以先在同一模型上找到努力等级曲线，再把不同模型加入矩阵比较，避免一次评估混入太多变量。

6.6 本章小结

本章覆盖 00:13:33–00:16:07。演讲者给出的核心方法，是用真实困难任务建立评估集，并在不同努力水平下测试 Claude 的表现。努力等级提升通常会改善能力，但高档位附近会出现边际收益递减；最高档可能消耗更多 token 和时间，却只带来很小改善。

低努力也有使用价值。它适合低智能需求、低延迟、高吞吐的任务，也可能在某些交互环境中诱导出快速捷径。Pokemon Red 例子说明，资源约束会改变模型策略；这种改变有时提高效率，有时带来短视。最终选择不能靠直觉，应当用质量、成本、延迟和失败类型共同校准。

7 模型大小与努力程度的组合决策

7.1 从完美评估退一步：先用经验法则落地

上一章把最可靠的方法讲清楚了：为真实难题建立评估集，在多个努力等级上跑一遍，再选择满足质量线的最低资源档位。问题在于，真实团队经常没有那么完整的开发环境。字幕在 00:16:07–00:16:23 处也直接承认，理想评估很难，许多客户没有完美的测试条件。因此本章的目标更加工程化：在评估暂时不充分时，怎样先选一个可落地的模型与努力等级组合。

可以把决策拆成两个旋钮。第一个旋钮是模型选择 M ，代表 Haiku、Sonnet、Opus 这类不同规模或不同能力段的模型家族。第二个旋钮是努力等级 E ，代表模型在推理时愿意花多少内部计算、令牌和时间。二者共同影响结果质量 Q 、延迟 T 和成本 C ：

$$Q \approx f(M, E, B_{\text{tok}}, \text{task}, \text{context}), \quad C, T \uparrow \text{ as } E \text{ and generated tokens increase.}$$

- Q : 任务成功率或输出质量，例如代码能否通过测试、摘要是否保留关键事实、仿真是否像真实系统。
- M : 模型家族或模型大小。字幕中的“海库”“喷呐”“欧普斯”按合同统一写作 Haiku、Sonnet、Opus；具体版本号需要画面确认。
- E : 努力等级，包括低、中、高、超高、最大等档位。
- B_{tok} : 令牌预算，限制模型可花费的推理和输出空间。
- T : 端到端等待时间。
- C : 运行成本，通常随令牌数和工作时长上升。

这个式子只是讲义中的抽象地图，视频没有给出精确函数。它的价值在于提醒读者：提高努力等级与换更强模型都可能提高质量，但它们消耗的资源形态不同，适用场景也不同。

7.2 努力等级阶梯：从最大努力到低努力

00:16:23 起，演讲者按照从右到左的方式解释努力等级。可将这段话整理成一个五档阶梯。

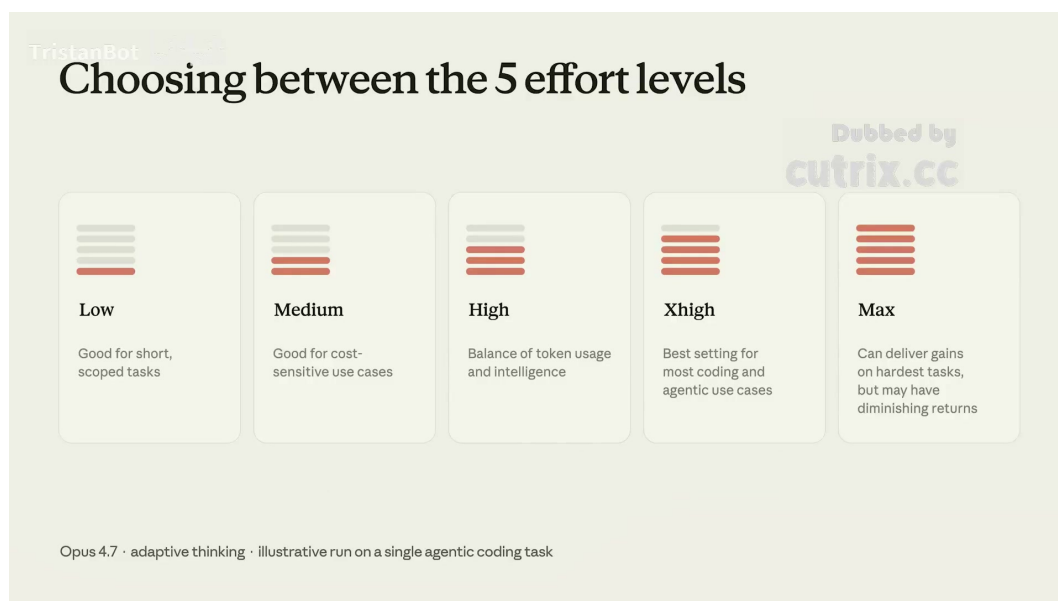


图 16: 五档努力等级把任务从低延迟需求到最难任务设置成一条实践阶梯：低努力适合分类、摘要、抽取等延迟敏感任务，最大努力留给明确困难且值得长时间推理的任务。¹⁶

努力等级的默认选择

如果缺少完整评估，先把“超高努力”视为通用起点。演讲者说明，这是 Claude Code 和 Claude 产品中采用的默认取向，因为它在智能、速度与令牌数量之间给出较稳的平衡。只有在确认任务极难、需要长时间推理时，才把“最大努力”作为首选。

努力等级	适用信号	主要代价与注意点
最大努力	任务非常困难，且明确需要 Claude 长时间思考。	最可能提升最难任务的表现，但边际收益递减；不宜作为日常默认档。
超高努力	对质量有要求，但团队暂时没有完整评估。	视频称其为产品默认值，适合作为“拿不准时”的起点。
高努力	任务需要一定智能，仍希望控制令牌消耗。	在令牌消耗和智能之间仍有较好平衡，速度通常快于更高档位。
中等努力	质量要求中等，成本或等待时间开始重要。	用来调节令牌消耗，适合较稳定、风险较低的任务。
低努力	分类、摘要、数据提取等延迟敏感任务。	以速度换取较浅的推理深度；用于问题结构清楚、容错空间较大的场景。

这张阶梯表的关键，不在于把每个任务机械塞进某一档。更实用的做法是先看任务的失败代价：如果错误会导致回滚、误判客户、生成错误代码或漏掉关键事实，就不应急着降低努力等级；如果任务像批量分类、字段抽取、格式归一化那样结构明确，低努力往往已经够用。

最大努力的常见误用

“最大努力”听起来最安全，实际会更贵、更慢，还可能只带来很小的增益。00:16:33–00:16:45 的建议很直接：除非已经确认任务特别困难，并且知道 Claude 需要较长时间思考，否则不要从最大档开始。

7.3 小模型高努力，还是大模型低努力

努力等级控制测试时计算的分配。模型大小对应训练阶段已经沉淀进模型的能力。00:17:31–00:17:40 中，演讲者把这两个杠杆并列起来：一个是训练时计算，一个是测试时计算。随后的问题很自然：应当使用 Haiku 这类小模型并提高努力程度，还是使用更强模型并降低努力程度？

视频给出的例子仍然回到前面的交通灯仿真。00:18:02–00:18:22 中，左侧是此前较强模型生成的交通灯仿真，右侧是 Haiku 家族模型生成的仿真。字幕说，Haiku 生成所用时间约为一半，令牌用量却相同，成品效果差距明显。演讲者甚至补了一句直观判断：他几乎看不出画面里的对象是否是车。

为什么大模型低努力常常更划算

大模型像一位经验更深的工程师，低努力时也能调用更丰富的先验和更稳的抽象能力；小模型像一位执行速度快的助理，即使给它更多草稿纸，也可能缺少识别复杂结构的底层能力。提高努力等级可以让模型多想几步，却不能完全补上模型本身没有学到的表示能力。

因此，00:18:30–00:18:43 的结论可以整理为一条实用规则：只要任务需要一点“智能”，通常优先选择更强模型，即使努力等级设得较低。这里的“智能”没有神秘含义，可以落到几个可检查的信号上：

- 任务需要在多个约束之间做权衡，例如质量、成本、法规、用户体验同时存在。
- 输出需要组合多步证据，例如调试、代码审查、复杂研究、方案设计。
- 错误难以及时发现，例如生成看似流畅但事实错误的总结。

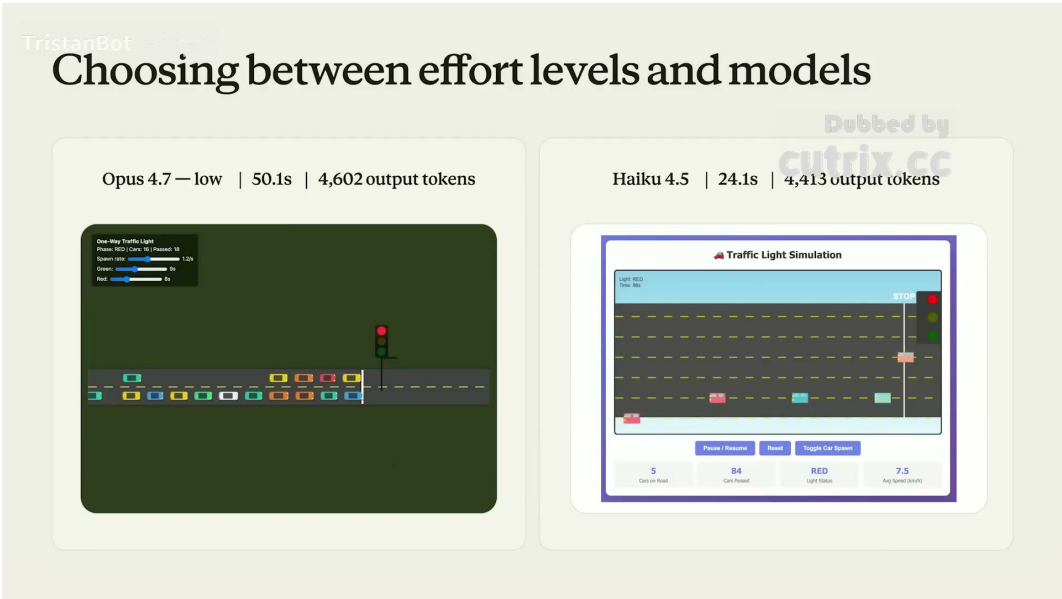


图 17：模型选择会与努力等级交互。画面对比了 Opus 低努力与 Haiku 在同一交通灯仿真任务上的结果：二者 token 量接近，但输出质量和可读性差距明显。¹⁷

- 任务目标需要模型自己澄清，例如“做得更专业”“找出根因”“给出可执行方案”。

小模型的优势在另一侧。00:18:45–00:19:02 中，演讲者建议把小模型留给智能需求不高、结果足够简单、用户不太在意细节差异的场景。典型例子包括大规模标签分类、固定 schema 的字段抽取、短文本改写、重复格式转换等。此时质量天花板由任务本身限定，强模型的额外能力可能转化成多余成本。

7.4 组合决策表：从质量线反推资源配置

将努力等级与模型大小放在同一张表里，实践判断会清晰很多。

任务画像	推荐起点	调整方向
高风险复杂任务：代码修复、长链研究、代理式操作、复杂规划。	更强模型 + 超高努力。	质量仍不足时升到最大努力；延迟不可接受时先降努力，再考虑拆分任务。
中等复杂任务：结构化总结、需求分析、常规问答增强。	Sonnet 家族或同级模型 + 高/超高努力。	若输出稳定，可降到中等努力；若遗漏关键约束，优先提高努力。
低智能高吞吐任务：分类、抽取、格式化、短摘要。	Haiku 家族或小模型 + 低/中等努力。	若误差集中在理解复杂语义，换更强模型比继续加努力更直接。
探索性任务：尚不清楚难度，也缺少评估。	更强模型 + 超高努力。	先收集失败案例，再建立小型评估集，后续再降成本。

这个表应当先从质量线读起。先问“什么样的答案算过关”，再问“过关答案允许花多少时间和令牌”。如果顺序倒过来，团队很容易为便宜配置写一套自我安慰的标准，最终把失败转移给下游人工返工。

7.5 本章小结

本章把 00:16:07–00:19:18 的经验法则整理为组合决策框架。努力等级控制测试时计算，模型大小代表训练阶段沉淀下来的能力。最大努力适合明确困难且高价值的任务；超高努力是缺少评估时的稳健默认；高努力适合仍需智能的常规任务；中等和低努力主要用于调节令牌消耗和延迟。

模型选择方面，视频的交通灯仿真对比给出一个朴素结论：只要任务需要明显理解、判断或多步规划，更强模型即使在较低努力下也常常更可靠。小模型适合低智能需求、高吞吐、结果简单且容错空间较大的工作。最终仍应回到评估：在 Haiku、Sonnet、Opus 等模型家族和多个努力等级之间做网格测试，选择满足质量线的最低成本组合。

8 总结与延伸：从手动杠杆到预算化任务

8.1 演讲者的最后清单：把思考能力留给模型

00:19:26–00:20:13 是演讲者给出的实操清单。它延续了前面几章的主线：思考能力应当尽量可用，推理空间应当足够，努力等级用来控制资源投入，评估用来校准平衡点。整理成行动项后，可以得到四条。



图 18: 收尾清单把整场讲座压缩为三条行动建议：启用思考并调节努力或预算，使用评估找到理想平衡，拿不准时选择 xHigh。¹⁸

四条可执行建议

1. 尽可能保持思考功能可用，让 Claude 有机会使用推理空间。

2. 通过草稿纸或等价机制，让模型知道自己可以使用思考工具，并通过努力等级控制投入。

3. 用真实困难任务做评估，因为简单题无法代表真正希望 Claude 完成的工作。

4. 拿不准时先选超高努力，这是演讲者认为在延迟、令牌数量和智能之间较高效的产品默认值。

这里的“草稿纸”可以理解为模型的临时工作台。人做复杂题时，会先写中间步骤、列约束、

排除错误路径；模型也需要类似空间来组织推理。若只允许它直接吐出最终答案，等于要求一个工程师不画草图、不列假设、不检查边界条件就提交方案。对于简单任务，这样做也许足够；对于复杂任务，失败往往会以遗漏条件、过早下结论、工具调用顺序错误的形式出现。

评估集要覆盖真正难题

00:19:50–00:19:58 强调，选择最难的评估通常有益，因为它能确认评估代表了你真正希望 Claude 完成的任务。只用简单样本评估，会让低成本配置显得过度乐观；上线后遇到长尾复杂案例，返工成本会迅速吞掉前期节省。

8.2 决策框架：质量线、预算、模型、工具

整场讲座可以压缩成一个四问框架。每次为任务选择 Claude 配置时，先按顺序回答这四个问题。

1. **质量线是什么？**需要的是“可读即可”的摘要，还是能通过测试的代码、可执行的研究结论、可审计的业务判断？质量线越高，越需要强模型、较高努力等级和更充分的工具上下文。
2. **可接受预算是多少？**预算至少包含三层：令牌预算 B_{tok} 、时间边界 B_{time} 、成本边界 B_{cost} 。同时观测实际耗时 T 与运行成本 C 。若任务有人工复核，预算还应包括返工成本。
3. **哪个模型与努力等级能过线？**缺少评估时，从更强模型加超高努力起步；有评估后，逐步降低努力等级或更换小模型，直到刚好触及质量线。
4. **是否需要工具或自适应思考？**如果任务需要查资料、跑代码、看文件、操作界面或长链验证，就要保留工具使用与自适应思考能力。只调努力等级无法替代外部信息入口。

从“开关思维”到“预算思维”

早期使用者容易把思考当成开关：开了就更聪明，关了就更快。更成熟的做法是预算思维：给任务设定质量标准、时间窗口、令牌上限和工具权限，让模型在约束内分配工作。开关只表达允许或禁止，预算表达目标和边界。

这个框架也解释了为什么“低努力小模型”未必总是省钱。若任务本身需要理解复杂语义，小模型可能输出大量看似完整但需要人工重做的答案。表面上 C 降低了，实际总成本会通过人工检查、二次调用、错误传播重新出现。反过来，简单任务使用强模型最大努力也会浪费资源，因为质量线已经很低，再多推理只是在平坦区间上增加令牌。

8.3 预算化长期任务：从一次回答到一段工作

00:20:13–00:21:00 是收尾中最有前瞻性的部分。演讲者设想的理想状态，是用户为 Claude 的工作设定标准和预算，无需为每次调用手动选择档位：一项长期任务可能持续一天或一周，用户告诉 Claude 预算是多少，或者期限是一周。随后，Claude 应当理解如何分配算力、消耗多少令牌、何时执行子任务，并在限制内返回结果。

这意味着“思考杠杆”的使用方式会从单次调用扩展到任务级管理。单次调用关注一次回答要不要多想；任务级管理关注整段工作怎样分配资源。一个长期研究任务可能包含检索资料、筛选证据、写初稿、发现冲突、补充验证、形成结论等阶段。每个阶段的重要性不同，风险不同，所需工具也不同。理想的 Claude 需要识别哪些子任务至关重要，把预算集中到高风险节点，而把低风险步骤用较低努力完成。

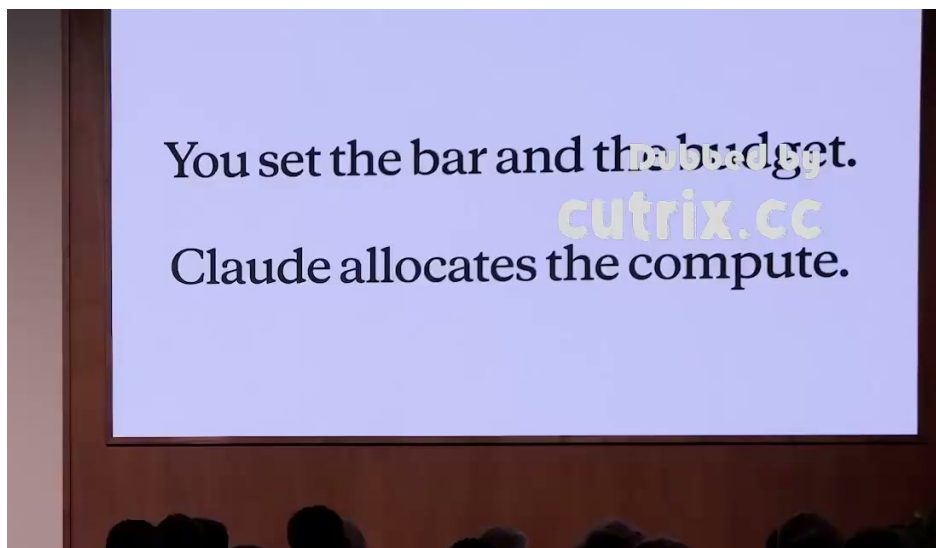


图 19: 最后一页给出任务级预算的操作原则：用户设定质量标准和预算，Claude 在这些边界内分配计算资源。¹⁹

可以用一个简单的资源分配模型理解这种变化：

$$B_{\text{task}} = B_{\text{tok}} + B_{\text{time}} + B_{\text{cost}} + B_{\text{tool}} + B_{\text{review}}$$

- B_{task} : 任务预算，代表整段工作的总约束。
- B_{tok} : 令牌预算，限制推理与输出空间。
- B_{time} : 时间预算，限制任务可运行的持续时间。
- B_{cost} : 成本预算，限制任务可消耗的费用或计费额度。
- B_{tool} : 工具预算，限制搜索、执行、读取、调用外部系统的次数或范围。
- B_{review} : 复核预算，代表人工检查、模型自检或交叉验证所需资源。

这个式子同样是讲义抽象，目的是帮助读者把“预算”从单一 token 上限扩展为任务治理。真正的系统会更复杂：某些工具调用成本很低却能显著降低幻觉，某些自检会增加时间但减少上线风险，某些中间结果值得保存给后续调用复用。重点在于，预算既用于控制成本，也用于把资源送到最该花的地方。

8.4 给读者的落地流程

最后，把整场讲座转成一个可执行流程：

1. 写下任务的质量线：必须正确、基本正确、可读即可，三者对应完全不同的配置。
2. 判断失败代价：错误是否会进入生产、影响用户、误导决策、造成返工。
3. 给出预算边界：最长等待时间、可接受令牌量、可接受费用、是否允许工具。
4. 选择初始配置：复杂任务从强模型加超高努力开始；简单高吞吐任务从小模型加低或中等努力开始。

5. 收集失败样本：把模型答错、漏掉、过慢、过贵的案例放进评估集。
6. 扫描配置矩阵：比较模型家族与努力等级，找到刚好达到质量线的组合。
7. 把配置写成策略：不同任务类型使用不同默认值，并保留升级路径。

最终 takeaway

思考杠杆的成熟用法，是把模型能力、努力等级、工具使用和任务预算放进同一个决策面板。质量线决定最低能力要求，预算决定可花资源，评估负责校准默认值，自适应思考负责在任务内部重新分配算力。

8.5 本章小结

本章保留了演讲者的实质性收尾：尽量让 Claude 保持思考能力，给它足够推理空间，用困难评估寻找平衡点，拿不准时先选超高努力。进一步看，未来的关键方向是预算化长期任务：用户设定标准、时间、令牌和工具边界，Claude 学会识别重要子任务，并把资源分配到最能提升结果质量的位置。

贯穿全讲的核心判断很直接：测试时计算是一根可调杠杆，但它需要和模型大小、工具权限、质量标准、成本约束一起使用。只追求更低成本会牺牲复杂任务质量，只追求更高努力会浪费资源。更稳的做法，是先定义任务要达到的质量线，再用评估把模型与努力等级压到刚好够用的位置。

资料来源与产物说明

来源或产物	用途
Bilibili 视频: AI 模型的思考杠杆	报告主线、口播证据、关键帧和时间脚注。
本地元数据: <code>source/video.info.json</code>	标题、上传者、时长、发布日期、封面与格式信息。
本地字幕: <code>source/video.ai-zh.srt</code>	章节边界、口播内容与时间定位。英文字幕未暴露，故使用 ai-zh 字幕，并在写作时校正明显 ASR/翻译噪声。
本地视频: <code>source/video.mp4</code>	1080p 关键帧抽取和逐帧视觉检查。
封面图: <code>source/video.jpg</code>	PDF 首页封面。
全局契约: <code>outline_contract.md</code>	章节边界、术语表、符号表、子代理任务划分和一致性风险。
Figure reports: <code>reports/figure_report_*.md</code>	每张图的保留理由、拒绝窗口、画面内容与时间来源。